

Get

- 设备使用率是如何解释的：
设备利用率不足源于**底层神经网络的负载分配不均衡及其顺序依赖特性**
1. 负载分配不均衡：有些设备忙，有些设备很闲
负载指设备需要处理的**计算量（如矩阵乘法、激活函数）或数据量（如模型权重、中间特征）**。
场景一：朴素的MoE
一个专家对应一张卡，部分专家处理问题时，有一些专家空闲，又限制于最长时间的专家，所以出现了使用率不高的场景
场景二：参数不同的transformer
看似平均分配的按层分配，实际上可能因为层之间的参数不等而产生计算时间的不同，例如前后的FFN的参数不同。
 2. 顺序依赖性：前一个步骤之星之后下一个步骤才有对应的参数能够继续进行下去
神经网络有严格的先后执行顺序

前情概要

- 文章的目的旨在解决**神经网络的缩放（Neural Network Scaling）**（是指通过扩大“模型容量、训练数据量、计算资源投入”这三大核心要素）的过程之中面临的挑战，例如训练时硬件使用率不高，计算成本过高，并行困难等问题
- GShard 的价值就在于通过**条件计算与自动分片技术**，打破缩放过程中的效率与并行化障碍，让千亿级参数模型的高效训练成为可能。

挑战

1. 模型的参数比GPU or TPU的内存高出一个数量级不止
2. 现有简单拆分计算图之后并行的方式没有高设备使用率
3. 设备之间的通行复杂度过高

本笔记学习了文章之中关于MoE+transformer的知识点，至于多卡沟通，有兴趣再看

内容

前面我们已经学习过了，MoE+LSTM，知道MoE的大概思路。

模型容量增长但计算量亚线性增长的特定，能够很好解决计算开销问题。

融合transformer

传统 Transformer 层 (图 3a)	MoE-Transformer 层 (图 3b)
自注意力 → 层归一化 → 前馈层 → 残差连接	自注意力 → 层归一化 → MoE 层 → 残差连接
(全量参数激活)	(仅部分“专家”激活)

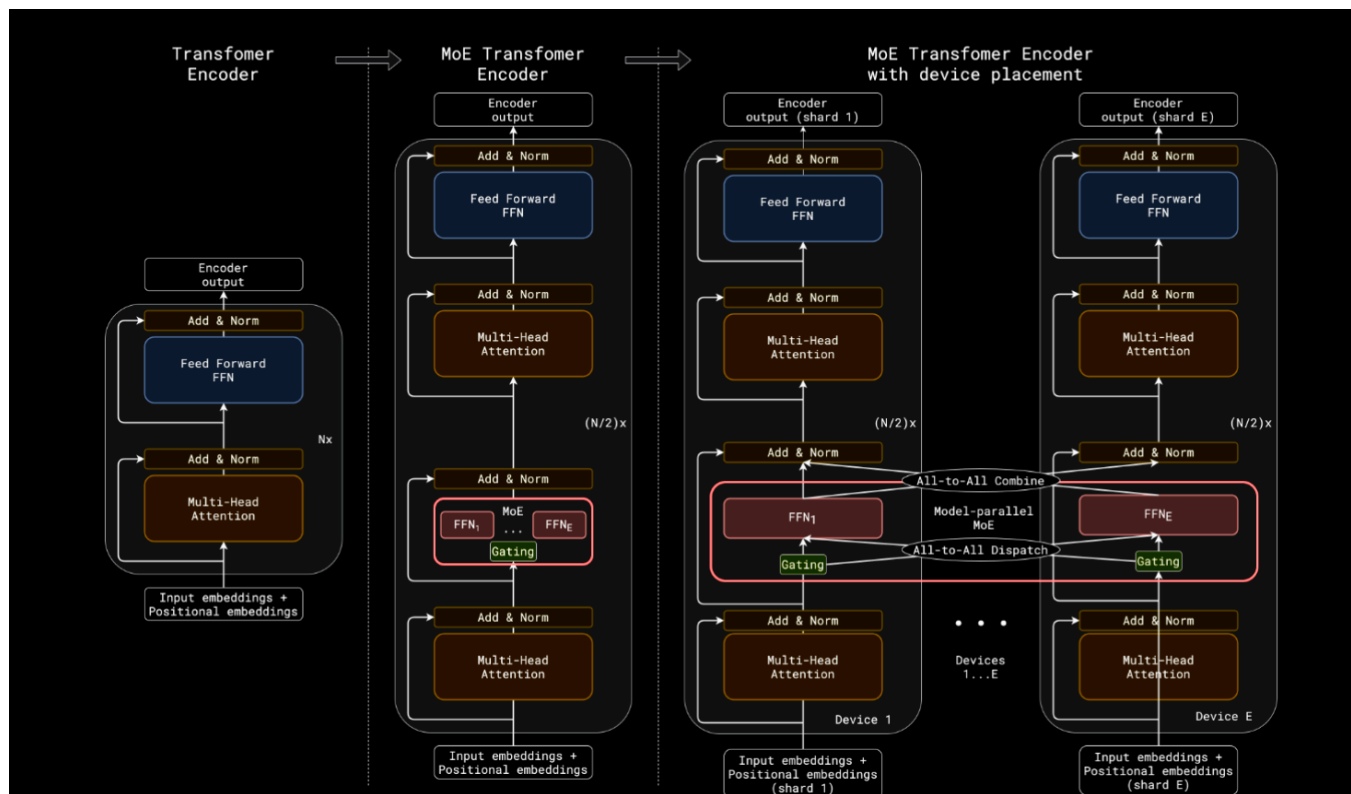


Figure 3: Illustration of scaling of Transformer Encoder with MoE Layers. The MoE layer replaces the every other Transformer feed-forward layer. Decoder modification is similar. (a) The encoder of a standard Transformer model is a stack of self-attention and feed forward layers interleaved with residual connections and layer normalization. (b) By replacing every other feed forward layer with a MoE layer, we get the model structure of the MoE Transformer Encoder. (c) When scaling to multiple devices, the MoE layer is sharded across devices, while all other layers are replicated.

如图，使用MoE代替传统的FFN，MoE之中包含了多个专家
 每一个专家都是双层全连接网络，和原FFN的结构一致
 具体计算办法：

1. 门控机制（核心）：

门控机制选择砖家决定了该模型是否能够正确训练和充分使用那么多的参数
 涉及到两个问题：专家之间的选择，被选择专家的权重比例。

2. topK选择：

最多选择2个专家

3. 加权融合

原文之中取消了正态分布的做法，直接使用了矩阵乘法对于专家打分
 再用softmax处理，得到归一化的分数

最后保证topk的求和为1，得到最终分数

创新

创新点集中在专家的选择上，为了保证参数使用的有效性，从**专家间选择**and**topK专家的平衡**两个角度来看问题

处理平衡问题（软硬皆施）

- 引入了**硬约束的专家容量限制**该观点：
 1. 给每个专家设置“最大处理 token 数”（容量 C ），若专家(e_1)的已处理 token 数($c_{e1} < C$)，才将 token 分配给它
 2. 若超过容量，该 token 仅通过残差连接传递，不占用专家资源
- 目的：直接残差连接使得部分专家不会过度训练，模型陷入局部最小值

专家容量计算公式：

$$C = \frac{2Samples}{Groups * Experts}$$

理解为，每一个专家在该组（G）样本处理之中，最多能够占用两倍平均的专家使用数实际上还有很好的收获：**平衡了多设备的计算问题！**

- 引入了**软约束的辅助损失**：（和原MoE思路相同）
计算公式

$$aux = \frac{1}{E} \sum_{e=1}^E \left(\frac{c_e}{S} \cdot m_e \right)$$

- 符号含义：
 - c_e ：专家 e 实际处理的 token 数；
 - S ：每组的 token 数（分组处理，1-41 中 Data 是 S 个 token 的组）；
 - m_e ：专家 e 的平均门控权重 $m_e = \frac{1}{S} \sum_{s=1}^S g_{s,e}$
- 依旧是那个思想，越平均，损失越小

随机路由

我们给出了计算手段

- **分配概率**： $2 * e_2$ ， 决定了次专家能否接触到token，和专家一协助
- **原始概率**： e_1 ， 决定了和专家一协助时候的权重

目的：

1. 专家之间有最大容量限制，如果全部由专家一和专家二共同处理，实际上是不能体现出每次计数的含金量的（专家二虽然弱势，但是仍然会被计数+1，这是不公平的）。专家一就是最有发言权的。因此，解决办法就是部分地方不设置专家二，也就是分配概率的由来。
2. 会打破计算亚线性增长的点，因为都是使用两个专家。